

Employee Database – Querying Data

DML, Clauses, Joins and Window Functions

Prepared By	Dhanalakshmi
Course	Data Analytics
Module	MySQL Assignment 2: Querying Data

Project Objective

This assignment builds on the Employee Database created earlier. With the tables, relationships, and constraints already in place, the focus here shifts to working with the data itself — inserting records and then querying them using clauses, aggregate functions, grouping, joins, and window functions to pull out meaningful insights.

Data Insertion & Consistency

- The Departments, Location, and Employees tables are populated with sample records.
- Every Department_id and Location_id used in the Employees table already exists in its parent table, so the foreign key relationships stay valid.

MySQL DML (Data Manipulation Language)

- DML is used to work with the data already sitting inside the tables — inserting, updating, and querying it.
- This assignment covers DISTINCT, WHERE, ORDER BY, LIMIT, aggregate functions, GROUP BY, HAVING, JOINS, and window functions.

Part A – Inserting the Data

The mentor-provided dataset is used here: 13 departments, 4 locations, and 30 employees. Departments and Location are populated first since Employees depends on both of them through foreign keys.

Insert into Departments (Department_id, Department_name) Values
(1, 'Software Development'),(2, 'Marketing'),(3, 'Data Science'),
(4, 'Human Resources'), ... (13, 'Operations');

Insert into Location (Location_name) Values
('Chennai'),('Bangalore'),('Hyderabad'),('Pune');

The Location table's column is Location_name, so that is the column referenced in the insert, rather than a plain 'location'.

Thirty employee records are then inserted, spread across the 13 departments and 4 locations.

Insert into Employees (Employee_id, Employee_name, Gender, Age, Hire_date, Designation, Department_id, Location_id, Salary) Values
(5001, 'Vihaan Singh', 'M', 27, '2015-01-20', 'Data Analyst', 3, 4, 60000),
(5002, 'Reyansh Singh', 'M', 31, '2015-03-10', 'Network Engineer', 12, 1, 80000),

...

(5030, 'Sara Iyer', 'F', 28, '2019-11-20', 'Project Manager', 5, 1, 90000);

(The full 30-row insert statement is in the attached .sql file — shortened here for readability.)

The screenshot shows the MySQL Workbench interface. The SQL editor at the top contains an insert statement for the 'Employees' table, followed by three select statements. The result grid below shows the first 10 rows of the 'Employees' table. The right sidebar displays 'Automatic context help is di for the current caret'.

SQL Editor:

```
52 (5026, 'Vivaan Singh', 'M', 29, '2019-03-10', 'Business Analyst', 3, 2, 75000),
53 (5027, 'Ananya Paul', 'F', 32, '2019-05-20', 'Content Writer', 6, 3, 60000),
54 (5028, 'Anaya Kapoor', 'F', 26, '2019-07-05', 'Event Coordinator', 6, 1, 60000),
55 (5029, 'Arjun Kumar', 'M', 33, '2019-09-11', 'Quality Assurance Analyst', 12, 2, 80000),
56 (5030, 'Sara Iyer', 'F', 28, '2019-11-20', 'Project Manager', 5, 1, 90000);
57
58 • Select * from Employees;
59 • Select * from Departments;
60 • Select * from Location;
```

Result Grid:

Employee_id	Employee_name	Gender	Age	Hire_date	Designation	Salary	Dept
5001	Vivaan Singh	M	27	2015-01-20	Data Analyst	60000.00	3
5002	Reyansh Singh	M	31	2015-03-10	Network Engineer	80000.00	12
5003	Aaradhya Iyer	F	26	2015-05-20	Customer Support Executive	45000.00	10
5004	Kiera Malhotra	F	29	2015-07-05	Business Development Executive	70000.00	8
5005	Anvi Chaudhary	F	25	2015-09-11	Business Development Executive	55000.00	11
5006	Dhruv Shetty	M	28	2015-11-20	UI Developer	65000.00	8
5007	Anushka Singh	F	32	2016-01-15	Marketing Manager	90000.00	2
5008	Diya Jha	F	27	2016-03-05	Graphic Designer	70000.00	8
5009	Kiaan Desai	M	30	2016-05-20	Sales Executive	55000.00	11

Part B – Clauses & Operators

DISTINCT Values

Removes duplicate salary figures so only the unique salary amounts are returned.

Select distinct Salary from Employees;

The screenshot shows the SQL Developer interface. The query window contains the following SQL code:

```

59 • Select * from Departments;
60 • Select * from Location;
61 • Select * from Employees;
62
63
64 /* CLAUSE & OPERATORS */
65
66 -- 1. DISTINCT VALUES: retrieve distinct salaries from the Employees table
67 • Select distinct Salary from Employees;

```

The result grid below the query window displays the following data:

Salary
60000.00
80000.00
45000.00
70000.00
55000.00
65000.00
90000.00
95000.00
75000.00
85000.00

Alias (AS)

Renames the Age and Salary columns in the output to Employee_Age and Employee_Salary for readability, without changing the underlying column names in the table.

Select Age as Employee_Age, Salary as Employee_Salary from Employees;

The screenshot shows the SQL Developer interface with the following SQL code in the query window:

```

63
64 /* CLAUSE & OPERATORS */
65
66 -- 1. DISTINCT VALUES: retrieve distinct salaries from the Employees table
67 • Select distinct Salary from Employees;
68
69 -- 2. ALIAS (AS): alias age and salary columns
70 • Select Age as Employee_Age,
71 Salary as Employee_Salary from Employees;

```

The result grid below the query window displays the following data:

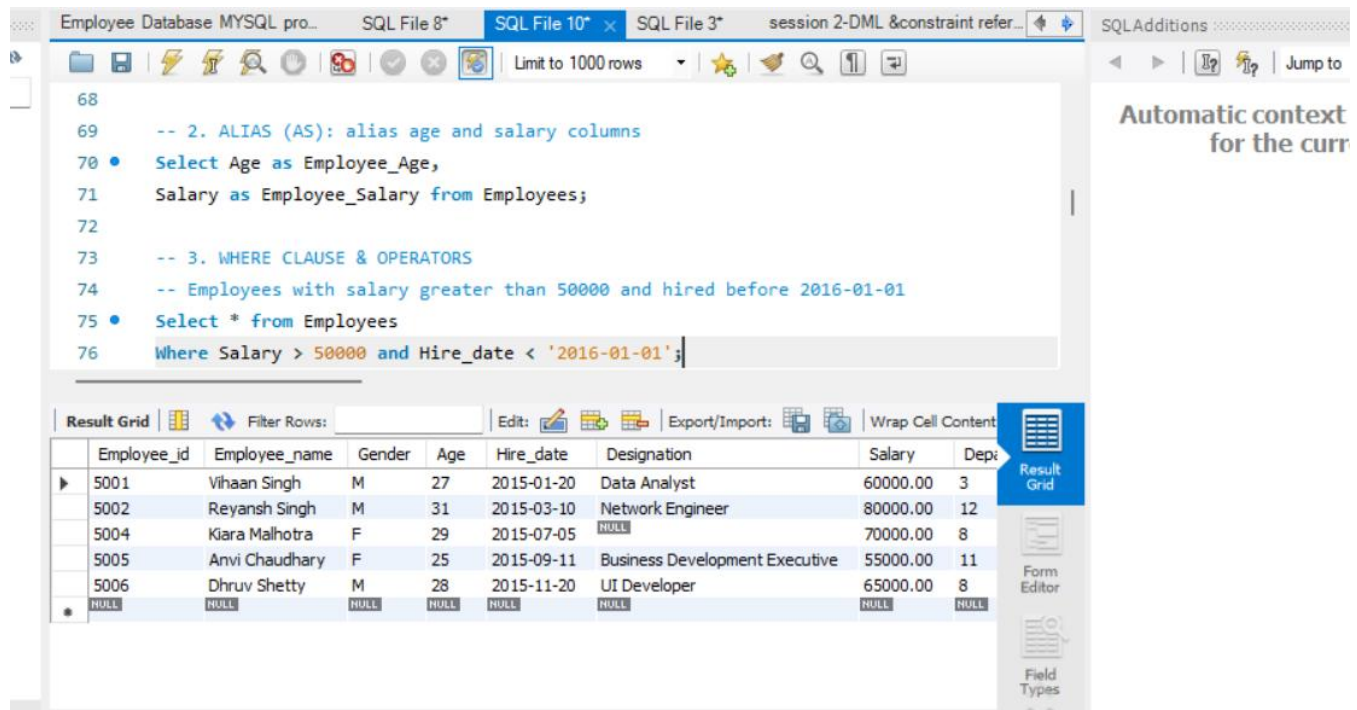
Employee_Age	Employee_Salary
27	60000.00
31	80000.00
26	45000.00
29	70000.00
25	55000.00
28	65000.00
32	90000.00
27	70000.00
30	55000.00
29	80000.00

WHERE Clause & Operators

Filters employees earning above ₹50,000 who joined before 1 January 2016.

Select * from Employees

Where Salary > 50000 and Hire_date < '2016-01-01';



The screenshot shows the MySQL Workbench interface. The top toolbar includes icons for file operations, execution, and navigation. The SQL editor contains the following queries:

```
68
69 -- 2. ALIAS (AS): alias age and salary columns
70 • Select Age as Employee_Age,
71     Salary as Employee_Salary from Employees;
72
73 -- 3. WHERE CLAUSE & OPERATORS
74 -- Employees with salary greater than 50000 and hired before 2016-01-01
75 • Select * from Employees
76 Where Salary > 50000 and Hire_date < '2016-01-01';
```

The bottom section displays the 'Result Grid' with the following data:

	Employee_id	Employee_name	Gender	Age	Hire_date	Designation	Salary	Dept
▶	5001	Vihaan Singh	M	27	2015-01-20	Data Analyst	60000.00	3
	5002	Reyansh Singh	M	31	2015-03-10	Network Engineer	80000.00	12
	5004	Kiara Malhotra	F	29	2015-07-05	NULL	70000.00	8
	5005	Anvi Chaudhary	F	25	2015-09-11	Business Development Executive	55000.00	11
	5006	Dhruv Shetty	M	28	2015-11-20	UI Developer	65000.00	8
•	NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL

On the right side, there is a sidebar with 'Automatic context for the curr' and a 'Result Grid' button. Below the result grid, there are buttons for 'Form Editor' and 'Field Types'.

One employee record was left without a designation on purpose. UPDATE is used here to fill that gap, since a plain SELECT can only find it — not fix it.

Update Employees

Set Designation = 'Data Scientist'

Where Designation is null;

Employee Database MYSQL pro... SQL File 8* SQL File 10* x SQL File 3* session 2-DML &constraint refer...

Limit to 1000 rows

```

75 • Select * from Employees
76 Where Salary > 50000 and Hire_date < '2016-01-01';
77
78 -- Find the employee whose designation is missing and fill it with "Data Scientist"
79 • Update Employees
80 Set Designation = 'Data Scientist'
81 Where Designation is null;
82
83 • Select * from Employees;

```

Result Grid | Filter Rows: | Edit: | Export/Import: | Wrap Cell Content

	Employee_id	Employee_name	Gender	Age	Hire_date	Designation	Salary	Dept
▶	5001	Vihaan Singh	M	27	2015-01-20	Data Analyst	60000.00	3
	5002	Reyansh Singh	M	31	2015-03-10	Network Engineer	80000.00	12
	5003	Aaradhya Iyer	F	26	2015-05-20	Customer Support Executive	45000.00	10
	5004	Kiara Malhotra	F	29	2015-07-05	Data Scientist	70000.00	8
	5005	Anvi Chaudhary	F	25	2015-09-11	Business Development Executive	55000.00	11
	5006	Dhruv Shetty	M	28	2015-11-20	UI Developer	65000.00	8
	5007	Anushka Singh	F	32	2016-01-15	Marketing Manager	90000.00	2
	5008	Diya Jha	F	27	2016-03-05	Graphic Designer	70000.00	8
	5009	Kiaan Desai	M	30	2016-05-20	Sales Executive	55000.00	11

Result Grid
Form Editor
Field Types

Part C – Sorting and Grouping Data

ORDER BY

Sorts employees by Department_id from lowest to highest, and within each department by salary from highest to lowest.

Select * from Employees

Order by Department_id asc, Salary desc;

Employee Database MYSQL pro... SQL File 8* SQL File 10* SQL File 3* session 2-DML & constraint refer...

Limit to 1000 rows

SQLAdditions

Automatic context help is disabled. Use the for the current caret position or to t

```

82
83 • Select * from Employees;
84
85
86 /* SORTING AND GROUPING DATA */
87
88 -- 1. ORDER BY: department ID ascending, salary descending
89 • Select * from Employees
90 Order by Department_id asc, Salary desc;

```

Result Grid

Employee_id	Employee_name	Gender	Age	Hire_date	Designation	Salary	Departm
5020	Aarav Nair	M	32	2018-03-05	Software Engineer	90000.00	1
5007	Anushka Singh	F	32	2016-01-15	Software Engineer	90000.00	2
5023	Anika Paul	F	30	2018-09-20	Public Relations Specialist	70000.00	2
5011	Saanvi Patel	F	28	2016-09-20	Marketing Analyst	60000.00	2
5025	Avni Iyengar	F	31	2019-01-20	Data Scientist	100000.00	3
5026	Vivaan Singh	M	29	2019-03-10	Business Analyst	75000.00	3
5001	Vihaan Singh	M	27	2015-01-20	Data Analyst	60000.00	3
5016	Zoya Khan	F	31	2017-07-05	Legal Counsel	100000.00	4
5022	Aadhya Iyengar	F	28	2018-07-10	HR Specialist	60000.00	4

Employees 7 x

Output

Action Output

#	Time	Action	Message
50	00:18:10	Update Employees Set Designation = 'Data Scientist' Where Designation is null	1 row(s) affected Rows matched: 1 Changed: 1 Warnings: 0
51	00:18:18	Update Employees Set Designation = 'Data Scientist' Where Designation is null	0 row(s) affected Rows matched: 0 Changed: 0 Warnings: 0
52	00:18:21	Use employee	0 row(s) affected
53	00:18:21	Insert into Departments (Department_id, Department_name) Values (1, 'Software Development'), (2, 'Marketing'...	Error Code: 1062. Duplicate entry '1' for key 'departments.PRIMARY'
54	00:18:36	Select * from Employees LIMIT 0, 1000	30 row(s) returned

LIMIT

Pulls out only the first 5 employees who joined in 2018, ordered by their hire date.

```

Select * from Employees
Where Year(Hire_date) = 2018
Order by Hire_date asc
Limit 5;

```


Employee Database MySQL pro... SQL File 8 SQL File 10 x SQL File 3 session 2-DML &constraint refer... SQLAdditions

Limit to 1000 rows

```

88 -- 1. ORDER BY: department ID ascending, salary descending
89 • Select * from Employees
90 Order by Department_id asc, Salary desc;
91
92 -- 2. LIMIT: first 5 employees hired in the year 2018
93 • Select * from Employees
94 Where Year(Hire_date) = 2018
95 Order by Hire_date asc
96 Limit 5;

```

Result Grid

	Employee_id	Employee_name	Gender	Age	Hire_date	Designation	Salary	Departme
▶	5019	Ishika Patel	F	29	2018-01-15	Talent Acquisition Specialist	55000.00	4
	5020	Aarav Nair	M	32	2018-03-05	Software Engineer	90000.00	1
	5021	Advik Kapoor	M	26	2018-05-20	Finance Analyst	85000.00	7
	5022	Aadhya Iyengar	F	28	2018-07-10	HR Specialist	60000.00	4
	5023	Anika Paul	F	30	2018-09-20	Public Relations Specialist	70000.00	2
•	NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL

Result Grid
Form Editor
Field Tunes

Aggregate Functions

SUM adds up every salary in the Finance department (department_id 7 in this dataset) into a single total.

Select sum(Salary) as Total_Finance_Salary
From Employees

Where Department_id = 7;

The screenshot shows the SQL Developer interface with a query window and a results grid. The query is as follows:

```
94 Where Year(Hire_date) = 2018
95 Order by Hire_date asc
96 Limit 5;
97
98 -- 3. AGGREGATE FUNCTIONS
99 -- Sum of all salaries in the Finance department (department_id = 7)
100 • Select sum(Salary) as Total_Finance_Salary
101 From Employees
102 Where Department_id = 7;
```

The results grid shows a single row with the value 170000.00 for the column Total_Finance_Salary.

The Output window shows the following messages:

#	Time	Action	Message
52	00:18:21	Use employee	0 row(s) affected
53	00:18:21	Insert into Departments (Department_id, Department_name) Values (1, 'Software Development'), (2, 'Marketing'...	Error Code: 1062. Duplicate entry '1' for key 'departments.PRIMARY'
54	00:18:36	Select * from Employees LIMIT 0, 1000	30 row(s) returned
55	00:19:18	Select * from Employees Order by Department_id asc, Salary desc LIMIT 0, 1000	30 row(s) returned
56	00:19:57	Select * from Employees Where Year(Hire_date) = 2018 Order by Hire_date asc Limit 5	5 row(s) returned
57	00:20:22	Select sum(Salary) as Total_Finance_Salary From Employees Where Department_id = 7 LIMIT 0, 1000	1 row(s) returned

MIN picks out the smallest age across the whole Employees table, identifying the youngest employee.

Select min(Age) as Min_age

From Employees;

The screenshot shows the SQL Developer interface with a query window and a results grid. The query is as follows:

```
98 -- 3. AGGREGATE FUNCTIONS
99 -- Sum of all salaries in the Finance department (department_id = 7)
100 • Select sum(Salary) as Total_Finance_Salary
101 From Employees
102 Where Department_id = 7;
103
104 -- Minimum age among all employees
105 • Select min(Age) as Min_age
106 From Employees;
```

The results grid shows a single row with the value 25 for the column Min_age.

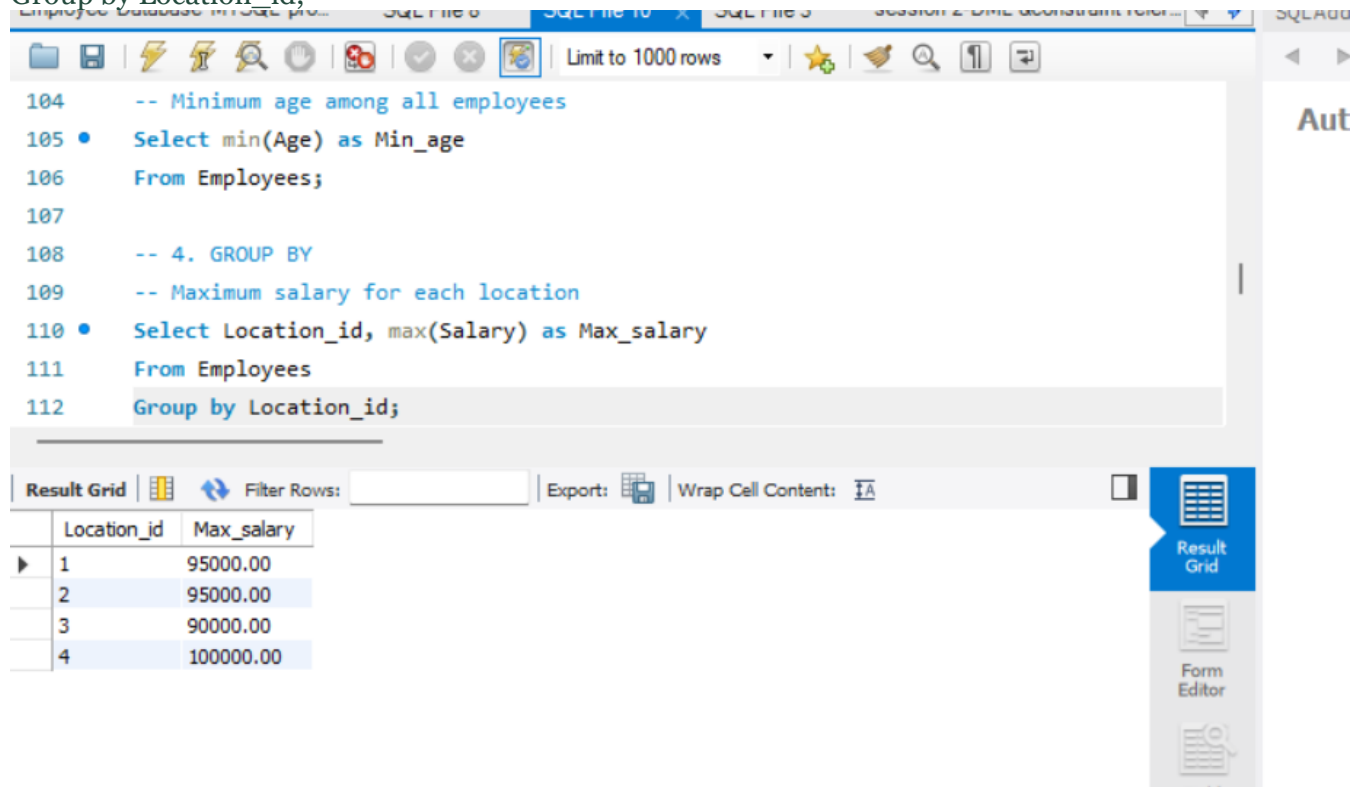
GROUP BY

Groups employees by Location_id and returns the highest salary found in each group.

Select Location_id, max(Salary) as Max_salary

From Employees

Group by Location_id;



The screenshot shows the SQL Developer interface. The SQL Editor contains the following code:

```
104 -- Minimum age among all employees
105 • Select min(Age) as Min_age
106 From Employees;
107
108 -- 4. GROUP BY
109 -- Maximum salary for each location
110 • Select Location_id, max(Salary) as Max_salary
111 From Employees
112 Group by Location_id;
```

The Results pane shows the output of the query, which is a table with two columns: Location_id and Max_salary. The table contains four rows of data:

Location_id	Max_salary
1	95000.00
2	95000.00
3	90000.00
4	100000.00

Filters to designations that contain the word 'Analyst', then groups by the exact designation to get an average salary for each one.

Select Designation, avg(Salary) as Avg_salary

From Employees

Where Designation like '%Analyst%'

Group by Designation;

The screenshot shows a SQL IDE with a query editor and a result grid. The query is as follows:

```
110 • Select Location_id, max(Salary) as Max_salary
111 From Employees
112 Group by Location_id;
113
114 -- Average salary for each designation containing the word 'Analyst'
115 • Select Designation, avg(Salary) as Avg_salary
116 From Employees
117 Where Designation like '%Analyst%'
118 Group by Designation;
```

The result grid shows the following data:

Designation	Avg_salary
Data Analyst	60000.000000
Marketing Analyst	60000.000000
Supply Chain Analyst	60000.000000
Financial Analyst	85000.000000
Finance Analyst	85000.000000
Business Analyst	75000.000000
Quality Assurance Analyst	80000.000000

HAVING

Counts employees per department, then keeps only the departments where that count is below 3 — HAVING is needed here instead of WHERE because the filter applies after the grouping and counting is done.

```
Select Department_id, count(*) as Total_employees
From Employees
Group by Department_id
Having count(*) < 3;
```

The screenshot shows a SQL IDE with a query editor and a result grid. The query is as follows:

```
117 Where Designation like '%Analyst%'
118 Group by Designation;
119
120 -- 5. HAVING
121 -- Departments with less than 3 employees
122 • Select Department_id, count(*) as Total_employees
123 From Employees
124 Group by Department_id
125 Having count(*) < 3;
```

The result grid shows the following data:

Department_id	Total_employees
1	1
5	2
6	2
7	2
9	1
11	2
13	1

Groups female employees by location and keeps only the locations where their average age comes out below 30.

```
Select Location_id, avg(Age) as Avg_female_age
From Employees
Where Gender = 'F'
Group by Location_id
Having avg(Age) < 30;
```

L26

L27 -- Locations with female employees whose average age is below 30

L28 • Select Location_id, avg(Age) as Avg_female_age

L29 From Employees

L30 Where Gender = 'F'

L31 Group by Location_id

L32 Having avg(Age) < 30;

L33

L34

Result Grid		Filter Rows:	Export:	Wrap Cell Content:
Location_id	Avg_female_age			
1	26.7500			
2	27.3333			
4	29.2000			

Part D – Joins

Inner Join

Matches every employee to their department, returning only employees who currently have a department assigned.

```
Select e.Employee_name, e.Designation, d.Department_name
From Employees e
Inner join Departments d on e.Department_id = d.Department_id;
```

```

135  /* JOINS */
136
137  -- 1. INNER JOIN: employee names, designations, and department names
138  • Select e.Employee_name, e.Designation, d.Department_name
139    From Employees e
140    Inner join Departments d on e.Department_id = d.Department_id;

```

Employee_name	Designation	Department_name
Anvi Chaudhary	Business Development Executive	Business Development
Kiaan Desai	Sales Executive	Business Development
Ananya Paul	Content Writer	Content Creation
Anaya Kapoor	Event Coordinator	Content Creation
Aaradhya Iyer	Customer Support Executive	Customer Support
Arnav Rao	Customer Success Manager	Customer Support
Vihaan Mohan	Supply Chain Analyst	Customer Support
Vihaan Singh	Data Analyst	Data Science
Avni Iyengar	Data Scientist	Data Science
Vivaan Singh	Business Analyst	Data Science

Left Join

Keeps every department in the result regardless of whether it has employees or not — with LEFT JOIN, a department with zero matching employees would still show up with a count of 0 instead of disappearing from the results.

```

Select d.Department_id, d.Department_name, count(e.Employee_id) as Total_employees
From Departments d
Left join Employees e on d.Department_id = e.Department_id
Group by d.Department_id, d.Department_name;

```

```

142  -- 2. LEFT JOIN: all departments with total employees in each
143  • Select d.Department_id, d.Department_name, count(e.Employee_id) as Total_employees
144    From Departments d
145    Left join Employees e on d.Department_id = e.Department_id
146    Group by d.Department_id, d.Department_name;
147
148  -- 3. RIGHT JOIN: all locations with employee names, NULL where no employees
149  • Select l.Location_id, l.Location_name, e.Employee_name

```

Department_id	Department_name	Total_employees
11	Business Development	2
6	Content Creation	2
10	Customer Support	3
3	Data Science	3
8	Design	3
7	Finance	2
4	Human Resources	3
12	IT	4
2	Marketing	3
13	Operations	1

Automatic context help
for the current

Right Join

Keeps every location in the result regardless of whether it has employees or not — with RIGHT JOIN, a location with no matching employees would still appear, with NULL in the employee name column.

```
Select l.Location_id, l.Location_name, e.Employee_name
From Employees e
Right join Location l on l.Location_id = e.Location_id;
```

The screenshot shows a database query editor with a SQL query and its results. The query is as follows:

```
144 From Departments d
145 Left join Employees e on d.Department_id = e.Department_id
146 Group by d.Department_id, d.Department_name;
147
148 -- 3. RIGHT JOIN: all locations with employee names, NULL where no employees
149 • Select l.Location_id, l.Location_name, e.Employee_name
150 From Employees e
151 Right join Location l on l.Location_id = e.Location_id;
152
```

The results are displayed in a table with the following columns: Location_id, Location_name, and Employee_name. The table shows 10 rows of data, where the first 8 rows represent Bangalore and the last 2 rows represent Chennai. The Employee_name column contains the names of the employees for each location, with NULL values for locations that do not have any employees.

Location_id	Location_name	Employee_name
2	Bangalore	Aaradhya Iyer
2	Bangalore	Dhruv Shetty
2	Bangalore	Myra Verma
2	Bangalore	Vihaan Mohan
2	Bangalore	Kabir Nair
2	Bangalore	Anika Paul
2	Bangalore	Vivaan Singh
2	Bangalore	Arjun Kumar
1	Chennai	Reyansh Singh
1	Chennai	Anvi Chaudhary

The interface includes a 'Result Grid' tab, a 'Filter Rows' field, and an 'Export' button. The status bar at the bottom indicates 'Result 17' and 'Read Only'.

Cross Join

Pairs every department with every location, generating every possible combination between the two tables regardless of whether an employee actually exists there.

```
Select d.Department_name, l.Location_name
From Departments d
Cross join Location l;
```

Employee Database MYSQL pro... SQL File 8* SQL File 10* x SQL File 3* session 2-DML & constraint refer...

SQL File 10* x Limit to 1000 rows

150 From Employees e
 151 Right join Location l on l.Location_id = e.Location_id;
 152
 153 -- 4. CROSS JOIN: all combinations of departments and locations
 154 • Select d.Department_name, l.Location_name
 155 From Departments d
 156 Cross join Location l;
 157
 158 -- 5. SELF JOIN: pairs of employees in the same department, excluding self-pairs

Result Grid

Department_name	Location_name
Business Development	Pune
Business Development	Hyderabad
Business Development	Chennai
Business Development	Bangalore
Content Creation	Pune
Content Creation	Hyderabad
Content Creation	Chennai
Content Creation	Bangalore
Customer Support	Pune
Customer Support	Hyderabad

Result 18 x Read Only Context Help Snippets

Output
 Action Output

Self Join

Joins the Employees table to itself on Department_id, so colleagues in the same department are paired up, while the condition on Employee_id stops an employee from being paired with themselves.

Select e1.Employee_name as Employee_1, e2.Employee_name as Employee_2, e1.Department_id
 From Employees e1
 Join Employees e2
 On e1.Department_id = e2.Department_id
 And e1.Employee_id <> e2.Employee_id;

Employee Database MYSQL pro... SQL File 8* SQL File 10* x SQL File 3* session 2-DML & constraint refer...

SQL File 10* x Limit to 1000 rows

156 Cross join Location l;
 157
 158 -- 5. SELF JOIN: pairs of employees in the same department, excluding self-pairs
 159 • Select e1.Employee_name as Employee_1, e2.Employee_name as Employee_2, e1.Department_id
 160 From Employees e1
 161 Join Employees e2
 162 On e1.Department_id = e2.Department_id
 163 And e1.Employee_id <> e2.Employee_id;
 164

Result Grid

Employee_1	Employee_2	Department_id
Vihaan Singh	Avni Iyengar	3
Vihaan Singh	Vivaan Singh	3
Reyansh Singh	Atharv Yadav	12
Reyansh Singh	Kabir Nair	12
Reyansh Singh	Arjun Kumar	12
Aaradhya Iyer	Arnav Rao	10
Aaradhya Iyer	Vihaan Mohan	10
Kiara Malhotra	Dhruv Shetty	8
Kiara Malhotra	Diya Jha	8
Anvi Chaudhary	Kiaan Desai	11

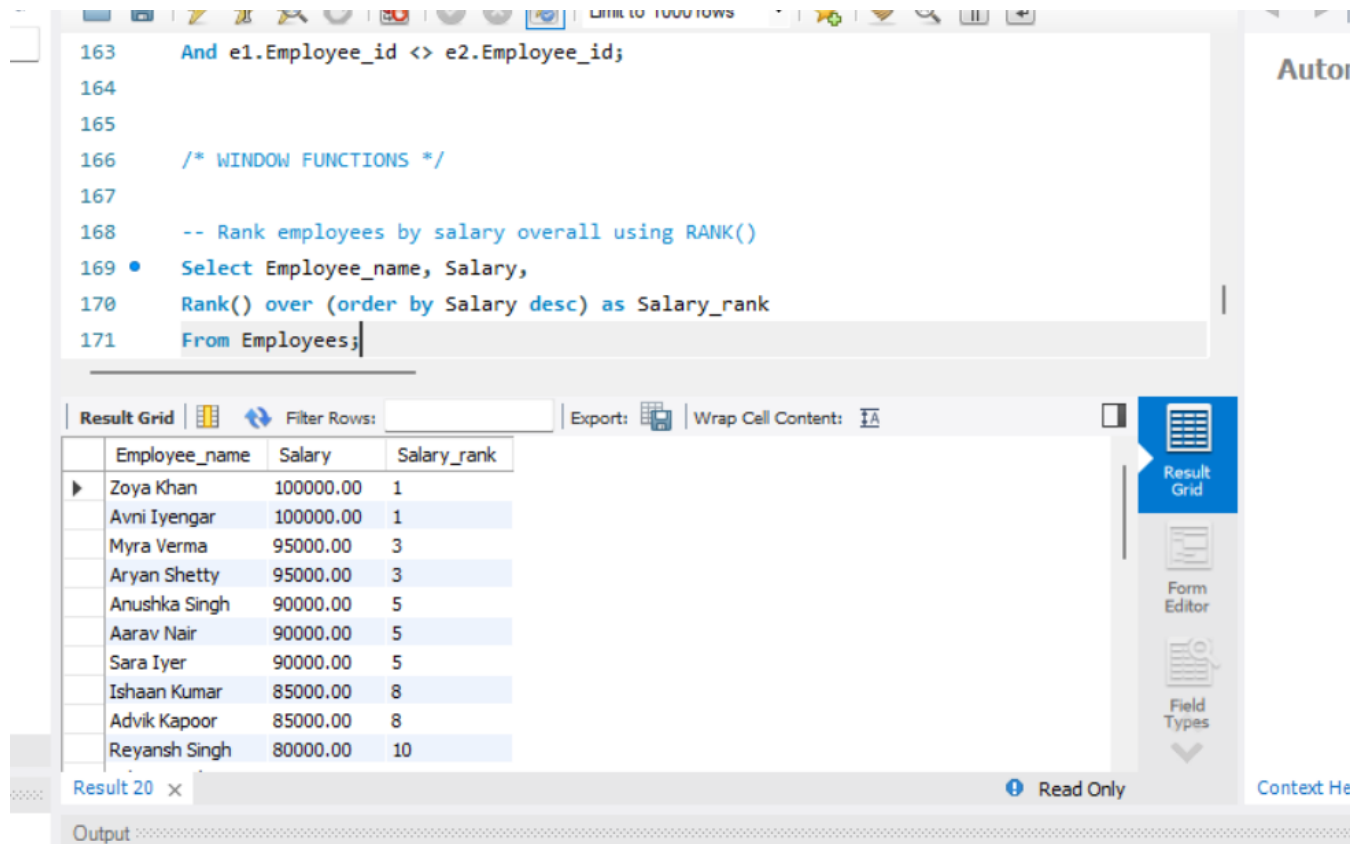
Result 19 x Read Only Context Help Snippets

Output
 Action Output

Part E – Window Functions

Ranks every employee by salary from highest to lowest across the whole company. RANK() gives tied salaries the same rank and then skips the next number.

```
Select Employee_name, Salary,  
Rank() over (order by Salary desc) as Salary_rank  
From Employees;
```



The screenshot shows a SQL IDE interface. The top pane contains a SQL query. The bottom pane displays the results in a grid format. The query is as follows:

```
163 And e1.Employee_id <> e2.Employee_id;  
164  
165  
166 /* WINDOW FUNCTIONS */  
167  
168 -- Rank employees by salary overall using RANK()  
169 • Select Employee_name, Salary,  
170 Rank() over (order by Salary desc) as Salary_rank  
171 From Employees;
```

The results grid shows the following data:

Employee_name	Salary	Salary_rank
Zoya Khan	100000.00	1
Avni Iyengar	100000.00	1
Myra Verma	95000.00	3
Aryan Shetty	95000.00	3
Anushka Singh	90000.00	5
Aarav Nair	90000.00	5
Sara Iyer	90000.00	5
Ishaan Kumar	85000.00	8
Advik Kapoor	85000.00	8
Reyansh Singh	80000.00	10

Ranks employees by salary again, but this time restarts the ranking inside each department using PARTITION BY. DENSE_RANK() is used here so tied salaries share a rank without leaving a gap in the numbering.

```
Select Employee_name, Department_id, Salary,  
Dense_rank() over (partition by Department_id order by Salary desc) as Dept_salary_rank  
From Employees;
```

Employee Database MYSQL pro... SQL File 8" SQL File 10" x SQL File 3" session 2-DML &constraint reter... SQLAdditions

Limit to 1000 rows

```

168 -- Rank employees by salary overall using RANK()
169 • Select Employee_name, Salary,
170 Rank() over (order by Salary desc) as Salary_rank
171 From Employees;
172
173 -- Rank employees by salary within each department using DENSE_RANK()
174 • Select Employee_name, Department_id, Salary,
175 Dense_rank() over (partition by Department_id order by Salary desc) as Dept_salary_rank
176 From Employees;

```

Result Grid

Employee_name	Department_id	Salary	Dept_salary_rank
Aarav Nair	1	90000.00	1
Anushka Singh	2	90000.00	1
Anika Paul	2	70000.00	2
Saarvi Patel	2	60000.00	3
Avni Iyengar	3	100000.00	1
Vivaan Singh	3	75000.00	2
Vihaan Singh	3	60000.00	3
Zoya Khan	4	100000.00	1
Aadhya Iyengar	4	60000.00	2
Ishika Patel	4	55000.00	3

Result 21 x Read Only Context Help

Builds a running total of salary within each department, adding each employee's salary to the total of everyone before them in that department.

```

Select Employee_name, Department_id, Salary,
Sum(Salary) over (partition by Department_id order by Employee_id) as Running_total_salary
From Employees;

```

Employee Database MYSQL pro... SQL File 8* SQL File 10* x SQL File 3* session 2-DML &constraint refer... SQLAddition

Limit to 1000 rows

```

173 -- Rank employees by salary within each department using DENSE_RANK()
174 • Select Employee_name, Department_id, Salary,
175 DENSE_RANK() over (partition by Department_id order by Salary desc) as Dept_salary_rar
176 From Employees;
177
178 -- Running total salary by department
179 • Select Employee_name, Department_id, Salary,
180 Sum(Salary) over (partition by Department_id order by Employee_id) as Running_total_s:
181 From Employees;

```

Result Grid

	Employee_name	Department_id	Salary	Running_total_salary
▶	Aarav Nair	1	90000.00	90000.00
	Anushka Singh	2	90000.00	90000.00
	Saanvi Patel	2	60000.00	150000.00
	Anika Paul	2	70000.00	220000.00
	Vihaan Singh	3	60000.00	60000.00
	Avni Iyengar	3	100000.00	160000.00
	Vivaan Singh	3	75000.00	235000.00
	Zoya Khan	4	100000.00	100000.00
	Ishika Patel	4	55000.00	155000.00
	Aadhya Iyengar	4	60000.00	215000.00

Result Grid
Form Editor
Field Types

Conclusion

- Building on the DDL structure from Assignment 1, this assignment used DML to insert data into Departments, Location, and Employees while keeping every foreign key relationship valid.
- DISTINCT, aliasing, WHERE, ORDER BY, and LIMIT were used to retrieve, rename, filter, and control exactly which rows come back.
- Aggregate functions (SUM, MIN, AVG, MAX) combined with GROUP BY and HAVING made it possible to summarise the data and filter those summaries by department, location, and designation.
- INNER, LEFT, RIGHT, CROSS, and SELF joins were used to combine and compare records across the three related tables in different ways.
- Window functions — RANK(), DENSE_RANK(), and a running SUM() — added ranking and cumulative calculations without collapsing the individual employee rows.
- Together, these queries turn the raw employee data into the kind of grouped, ranked, and filtered results a Data Analyst would actually use to answer business questions.

Thank You!